

텀 프로젝트 보고서

< Physical AI Term Project >

과목명 : 피지컬AI

학 번 : 2024405002

이 름 : 김만기

- 목 차 -

1. 프로젝트 개요
 - 가. 과제 목표
 - 나. 기존 파이프라인에서 확인한 문제
2. Dataset Extension
 - 가. Grasp / Push demonstration 확장
 - 나. Cube / Sphere object 추가
 - 다. RLDS / TFDS dataset rebuild
3. LoRA Test 및 OpenVLA Server 실행
 - 가. Short LoRA test
 - 나. Inference server 연결
4. Code Improvement
 - 가. Baseline 실행 결과
 - 나. 7D-to-4DOF action mapping 개선
 - 다. Inference request 수 감소 및 timing log 추가
5. 실험 결과
 - 가. MuJoCo cylinder grasp-and-lift
 - 나. Multishape object task 결과
 - 다. 실제 RaccoonBot 실행 결과
6. 한계점 및 결론

1. 프로젝트 개요

가. 과제 목표

이번 프로젝트의 목표는 수업에서 제공된 RaccoonBot OpenVLA pipeline을 그대로 실행하는 데에서 끝내지 않고, 데이터셋과 실행 코드를 직접 수정하여 더 다양한 task와 object에 대해 실험해보는 것이다. 전체 흐름은 MuJoCo에서 demonstration을 만들고, 이를 RLDS/TFDS 형식으로 변환한 뒤 OpenVLA fine-tuning 또는 inference pipeline과 연결하는 구조이다.

본 프로젝트에서는 크게 두 가지를 중점적으로 진행하였다. 첫 번째는 기존 colored cylinder grasp 중심의 dataset을 확장하는 것이고, 두 번째는 OpenVLA server에서 받은 7D action을 RaccoonBot의 4DOF 구조에 맞게 더 안정적으로 실행하도록 client 쪽 action mapping을 수정하는 것이다.

과제 안내에서 dataset extension과 code improvement를 각각 요구했기 때문에, 단순히 실행만 되는 상태가 아니라 무엇을 바꾸었고, 그 결과가 어떻게 달라졌는지 before / after evidence를 남기는 것을 목표로 하였다.

나. 기존 파이프라인에서 확인한 문제

기본 예제는 주로 colored cylinder를 대상으로 한 grasp task로 구성되어 있었다. instruction도 대부분 grasp the {color} cylinder와 같이 단순한 형태였다. 따라서 object type, task type, language instruction 측면에서 확장 여지가 있었다.

또한 OpenVLA가 출력하는 action은 다음과 같은 7D 형태이다.

```
[dx, dy, dz, droll, dpitch, dyaw, gripper]
```

하지만 RaccoonBot은 실제로 full 6D end-effector pose를 그대로 제어하는 구조가 아니라, xyz 이동과 gripper 동작 중심으로 실행되는 4DOF 로봇이다. 따라서 7D action을 거의 그대로 실행하면 gripper가 적절한 시점에 닫히지 않거나, target 근처로 가더라도 lift까지 이어지지 않는 문제가 발생하였다. 실제 baseline 실행에서도 red / blue cylinder를 안정적으로 잡지 못하는 경우를 확인하였다.

2. Dataset Extension

가. Grasp / Push demonstration 확장

먼저 기존 cylinder 환경에서 task variation을 추가하였다. 기존에는 grasp 중심이었기 때문에, push task metadata를 추가하고 instruction template도 여러 개로 늘렸다. 예를 들어 다음과 같은 instruction을 사용하였다.

```
grasp the red cylinder
grab the green cylinder
pick up the blue cylinder
lift the yellow cylinder
push the red cylinder
move the green cylinder forward
```

확장한 dataset은 총 20개의 episode로 구성하였다. 색상별로는 red, blue, green, yellow가 각각 5개씩 나오도록 하였고, task count는 grasp 7개, push 13개로 구성되었다. 생성된 instruction은 총

14종류였다. 이 결과는 `project_evidence/summaries/extended_dataset_summary.json`에 저장하였다.

관련 수정 파일은 다음과 같다.

```
Mujoco/raccoon_grasp_push_extended_dataset.py
```

나. Cube / Sphere object 추가

과제 안내에서 object type extension도 예시로 제시되어 있었기 때문에, 추가 보강 단계에서 cylinder 외에 cube와 sphere도 추가하였다. 기존 XML을 바탕으로 `Raccoon_multishape_objects.xml`을 만들었고, object shape은 다음과 같이 구성하였다.

```
red: cylinder
blue: cube
green: sphere
yellow: cylinder
```

이를 위해 `client_improvements/create_multishape_xml.py`를 작성하였다. 이 스크립트는 기존 `Raccoon_colored_cylinder.xml`을 읽고 blue object의 geom type을 box로, green object의 geom type을 sphere로 바꾼 XML을 생성한다. 생성 결과는 `client_improvements/evidence/multishape_scene_summary.json`에도 요약해두었다.

다. RLDS / TFDS dataset rebuild

MuJoCo demonstration을 생성한 뒤에는 OpenVLA 학습 pipeline에서 사용할 수 있도록 intermediate dataset으로 변환하고 TFDS dataset을 다시 build하였다. TFDS builder에서는 기존 intermediate root가 아니라 확장 dataset 경로를 읽도록 `INTERMEDIATE_ROOT`를 수정하였다.

수정한 파일은 다음과 같다.

```
Mujoco/rlds_dataset_builder/raccoon_pick_place/raccoon_pick_place_dataset_builder.py
```

TFDS build 결과 train split은 18개, validation split은 2개로 생성되었다. 관련 로그는 아래 파일에 저장하였다.

```
project_evidence/logs/rlds_convert_extended.log
project_evidence/logs/tfds_build_extended.log
```

3. LoRA Test 및 OpenVLA Server 실행

가. Short LoRA test

확장 dataset이 실제 OpenVLA fine-tuning script에서 정상적으로 읽히는지 확인하기 위해 100 step short LoRA test를 진행하였다. full training에서 사용하는 30000 step을 모두 수행한 것은 아니며, 과제 시간과 서버 사용 상황을 고려해 pipeline 검증 목적의 짧은 실험으로 진행하였다.

처음에는 CUDA 관련 문제가 있었지만, 서버 재접속 이후 `torch.cuda.is_available()`이 정상적으로 True가 되었고 100 step 학습이 완료되었다. 학습 로그에서는 dataset statistics 계산, 100 step 진행, checkpoint 저장이 정상적으로 이루어진 것을 확인하였다.

관련 로그는 다음과 같다.

```
project_evidence/logs/lora_100step_retry.log
project_evidence/logs/opencvla_server_lora_checkpoint.log
```

나. Inference server 연결

OpenVLA server는 A100 서버에서 실행하고, 로컬 PC의 MuJoCo client는 SSH tunnel을 통해 연결하였다. 로컬에서는 다음과 같은 방식으로 server URL을 사용하였다.

```
http://127.0.0.1:8000
```

이후 MuJoCo viewer를 실행하면서 server에서 action을 받아오는 방식으로 baseline과 개선된 client를 비교하였다.

4. Code Improvement

가. Baseline 실행 결과

baseline client로 red / blue cylinder를 실행했을 때는 target을 안정적으로 grasp하지 못하였다. 특히 OpenVLA raw action log를 확인했을 때 gripper command가 계속 0에 가까운 경우가 많았다. 즉, end-effector가 물체 근처로 움직이더라도 gripper가 닫히지 않아 실제 grasp-and-lift로 이어지지 않았다.

관련 evidence는 다음 파일로 저장하였다.

```
client_improvements/evidence/baseline_red.log
client_improvements/evidence/baseline_blue.log
client_improvements/evidence/baseline_failed_red.png
client_improvements/evidence/baseline_failed_blue.png
```

나. 7D-to-4DOF action mapping 개선

OpenVLA의 7D action과 RaccoonBot의 4DOF 실행 구조가 맞지 않는 문제를 줄이기 위해, client 쪽에서 action mapping을 수정하였다. 먼저 action_postprocess.py를 작성하여 raw action과 processed action을 모두 기록하고, delta action clipping 및 smoothing을 적용할 수 있게 하였다.

그 다음 action_target_assist_client.py와 action_target_assist_client_v2.py를 작성하였다. 이 코드는 task를 한 번에 실행하지 않고 다음과 같은 단계로 나누어 실행한다.

lift 또는 grasp task:

```
approach -> descend -> close -> lift
```

push task:

```
approach -> descend -> push
```

이 방식은 OpenVLA inference request를 유지하면서도, 실제 RaccoonBot이 실행 가능한 xyz + gripper action으로 변환하는 방식이다. 완전히 임의 위치의 물체를 찾아서 집는 자율 시스템은

아니지만, RaccoonBot의 구조적 제한을 고려했을 때 baseline보다 훨씬 안정적으로 동작을 해석하고 실행할 수 있었다.

다. Inference request 수 감소 및 timing log 추가

추가 code improvement로 request_every_n_steps 옵션을 만들었다. 기존 방식은 매 step마다 server에 inference 요청을 보내지만, 개선된 v2 client에서는 3 step마다 한 번씩만 inference를 요청하고, 그 사이에는 staged action mapping을 따라 동작을 이어가도록 하였다.

blue cube lift 실험에서 비교한 결과는 다음과 같다.

```
every-step request: 28 steps, 28 requests, ■■ step time ■ 811.7 ms
every-3-step request: 28 steps, 10 requests, ■■ step time ■ 581.2 ms
```

따라서 같은 lift 성공을 유지하면서도 server request 수는 약 1/3 수준으로 줄었고, 평균 step time도 감소하였다. 또한 각 step마다 inference latency, motion execution time, total step time, raw action, assisted action, gripper command, object lift/push distance를 csv로 기록하였다. 이로 인해 system이 더 빠르고, 디버깅하기 쉽고, 실행 결과를 더 명확하게 해석할 수 있게 되었다.

5. 실험 결과

가. MuJoCo cylinder grasp-and-lift

target-assisted action mapping을 적용한 뒤 MuJoCo에서 red / blue cylinder grasp-and-lift를 수행하였다. red cylinder의 최대 lift는 약 0.0162 m, blue cylinder의 최대 lift는 약 0.0159 m로 기록되었다.

관련 evidence는 다음과 같다.

```
client_improvements/evidence/target_assist_red_success.csv
client_improvements/evidence/target_assist_blue_success.csv
```

이 결과를 통해 baseline에서 gripper가 닫히지 않던 문제를 action mapping을 통해 어느 정도 해결할 수 있음을 확인하였다.

나. Multishape object task 결과

cube와 sphere를 추가한 multishape scene에서도 실험을 진행하였다. 주요 결과는 다음과 같다.

```
blue cube lift: ■■, lift ■ 0.0120 m
green sphere push: ■■, ■■ ■ 0.0104 m
red cylinder push: ■ 0.0090 m ■■
green sphere lift: lift ■ ■■■■ ■■■■ ■■■ ■■
```

특히 blue cube lift는 새 object type에 대한 lift 성공 evidence로 사용할 수 있고, green sphere push는 sphere object에 대한 push task 성공 evidence로 볼 수 있다. 반면 green sphere lift는 실제 viewer에서 확인했을 때 lift 과정에서 떨어졌고, 로그상 lift 값도 작기 때문에 성공으로 해석하지 않았다.

관련 evidence는 다음 파일에 정리되어 있다.

```
client_improvements/evidence/v2_task_evidence_summary.json
client_improvements/evidence/v2_final_evidence_summary.json
client_improvements/evidence/v2_blue_cube_lift.csv
client_improvements/evidence/v2_green_sphere_push.csv
client_improvements/evidence/v2_red_cylinder_push_tuned.csv
client_improvements/evidence/v2_green_sphere_lift.csv
```

다. 실제 RaccoonBot 실행 결과

실제 RaccoonBot에서도 red cylinder grasp-and-lift를 수행하였다. 실제 실험에서는 안전성과 반복성을 위해 target cylinder를 로봇 기준 대략 $x = 0$ cm, $y = 17$ cm 위치에 두고 실행하였다.

실험 결과 close stage와 lift stage까지 진행되었고, gripper_cmd=1.0으로 gripper가 닫힌 것을 로그에서 확인할 수 있었다. 마지막 target lift 값은 약 0.0122 m로 기록되었다. 실제 실행 영상도 real_robot_red_grasp_lift.mp4로 저장하였다.

관련 evidence는 다음과 같다.

```
client_improvements/evidence/real_target_assist_red_success.csv
client_improvements/evidence/real_target_assist_red_success.log
client_improvements/evidence/real_robot_red_grasp_lift.mp4
```

다만 이 실제 로봇 실험은 fixed-layout 실험이다. 즉, 카메라가 임의 위치의 빨간 실린더를 실시간으로 찾아 잡는 방식은 아니고, 정해진 target pose를 기준으로 OpenVLA inference pipeline과 개선된 action mapping을 연결한 결과이다.

6. 한계점 및 결론

이번 프로젝트에서 dataset extension, TFDS rebuild, short LoRA test, code improvement, MuJoCo 결과, 실제 RaccoonBot 결과까지 전체 pipeline을 한 번 연결해볼 수 있었다. 특히 baseline에서는 gripper command와 실행 timing 문제가 있었고, 이를 staged action mapping과 timing/action log를 통해 확인하고 보정하였다.

과제의 code improvement 항목에 대해서는 7D-to-4DOF action mapping 개선, inference request 수 감소, timing/action log 추가를 수행하였다. 그 결과 system은 이전보다 더 빠르고, 더 명확하게 디버깅할 수 있으며, RaccoonBot 구조에 맞게 더 안정적으로 실행되었다고 볼 수 있다.

물론 한계도 있다. 실제 로봇 실험은 fixed-layout 환경에서 수행되었기 때문에, 임의 위치 물체 인식 및 grasp까지 해결한 것은 아니다. 또한 LoRA는 100 step short test만 진행했으므로 성능 향상을 위한 full training 결과라고 보기 어렵다. Sphere lift 역시 실패했기 때문에, object shape에 따라 grasp 안정성이 달라진다는 점도 확인하였다.

그래도 최종적으로는 기존 cylinder grasp 중심 pipeline을 넘어 push/lift task, cube/sphere object, 다양한 instruction, timing log, 실제 RaccoonBot 실행까지 연결하였다. 따라서 이번 프로젝트는 OpenVLA pipeline을 그대로 실행하는 것뿐 아니라, RaccoonBot의 실제 제약을 고려해 실행부를 개선하고 그 효과를 실험적으로 확인했다는 점에서 의미가 있다고 생각한다.